

词法-语法分析器使用

大概说一下完成情况

项目的前两部分目前是这样，实现了一个完整的编译器前端工具链，包括：

- **词法分析器**：将源程序转换为词法单元（Token）序列
 - ☒ 正则->NFA
 - ☒ DFA数据结构/类设计
 - ☒ NFA->DFA, DFA 最小化
 - ☒ 输出单词值和种别，报错信息
- **语法分析器**：使用 SLR(1) 分析方法构建抽象语法树（AST）
 - ☒ 修改文法
 - ☒ 对词法传来的lex文件进行解析，建立 Token 映射
 - ☒ productions (项目集规范族的基础) 和 first/followSet 的构建
 - ☒ 构造 LR(0) 项目集与 SLR 分析表(Action/Goto Table)
 - ☒ 语法分析驱动程序 (Parser Driver)：分析输出规约过程
 - ☒ 规约序列提取：从上一步的里面提取老师要求的格式的规约序列

目录结构

```
Finallab/
├── lex/                                # 词法分析器目录
│   ├── Lex_Analysis.cpp               # 词法分析器主程序
│   ├── Lex_Analysis.exe               # 编译后的可执行文件
│   ├── test/                           # 测试用例目录
│   │   ├── test1.sy ~ test4.sy        # 测试源文件
│   │   └── output/                    # 词法分析输出目录
│   │       ├── lex1.txt ~ lex4.txt     # 词法分析结果（作业要求）
│   │       └──
│   └──
├── Syntax/                             # 语法分析器目录
│   ├── main.cpp                       # 语法分析器主程序
│   ├── GrammarAnalyzer.cpp/h          # 文法分析器
│   ├── SLRTable.cpp/h                 # SLR 表构建
│   ├── Parser.cpp/h                   # 语法分析器核心
│   ├── ReductionSequenceLogger.cpp/h  # 规约序列记录器
│   ├── SyntaxTree.h                   # 语法树结构
│   ├── compiler_test.exe               # 编译后的可执行文件
│   └── output/                         # 语法分析输出目录
│       ├── 1out.txt ~ 4out.txt         # 完整分析日志
│       ├── syntax1.txt ~ syntax4.txt   # 规约序列（作业要求）（可以用于中间代码生成）
│       └── tree1.txt ~ tree4.txt        # 语法树（做中间代码生成需要遍历树结构，更建议用这个）
└── build.bat                           # windows 编译脚本
```

— run.bat	# windows 运行脚本
— Makefile	# Linux/Mac 构建文件
— 词法-语法器.md	# 本说明文档

快速开始

1.编译项目

Windows 系统

双击运行 `build.bat`，或在finallab目录下命令行中执行：

```
./build.bat
```

这将编译：

1. 词法分析器 → `lex/Lex_Analysis.exe`
2. 语法分析器 → `Syntax/compiler_test.exe`

Linux/Mac 系统

在项目根目录执行：

```
make all
```

或分别编译：

```
make lex      # 仅编译词法分析器
make grammar  # 仅编译语法分析器
```

2.运行完整分析流程

Windows 系统

双击运行 `run.bat`，或在命令行中执行：

```
run.bat
```

这将自动：

1. 运行词法分析器（处理 `lex/test/test*.sy` 文件）
2. 运行语法分析器（读取词法分析结果）
3. 生成所有输出文件

Linux/Mac 系统

```
make run-all
```

或分别运行：

```
make run-lex          # 仅运行词法分析器
make run-grammar      # 仅运行语法分析器（需要先运行词法分析）
```

单步调试

详情请见下面的配置与自定义部分。

输出文件说明

1. 词法分析输出

位置: `lex/output/lex1.txt` ~ `lex4.txt`

格式:

```
<token文本>    <类型, 编号>
```

示例:

```
int <KW,1>
main    <KW,5>
(    <SE,23>
)    <SE,24>
{    <SE,25>
```

错误标记:

```
^    <ERROR, 行号, 列号>
```

2. 语法分析输出

语法分析器会生成三类输出文件:

1. 完整分析日志 (`*out.txt`)

位置: `Syntax/output/1out.txt` ~ `4out.txt`

内容:

- 文法初始化信息
- FIRST 集和 FOLLOW 集

```
===== FIRST SETS =====
Program      : { eps int void const float }
CompUnit     : { eps int void const float }
UnitList     : { eps int void const float }
Unit         : { int void const float }
Decl         : { int const float }
ConstDecl    : { const }
VarDecl      : { int float }
ConstDefList : { ID }
BType        : { int float }
ConstDef     : { ID }
```

```

ConstInit      : { ID INT FLOAT + - ( }
VarDefList     : { ID }
VarDef         : { ID }
InitVal        : { ID INT FLOAT + - ( }
FuncDef        : { int void float }
FuncName       : { ID main }
OptFParams     : { eps int float }
FuncType       : { void }
FuncFParams    : { int float }
FuncFParam     : { int float }
Block          : { { }
BlockItemList  : { eps ID INT FLOAT int return const float if + - ( { ; }
BlockItem      : { ID INT FLOAT int return const float if + - ( { ; }
Stmt           : { ID INT FLOAT return if + - ( { ; }
OptExp         : { eps ID INT FLOAT + - ( }
Exp            : { ID INT FLOAT + - ( }
Cond           : { ID INT FLOAT + - ( }
LVal          : { ID }
PrimaryExp     : { ID INT FLOAT ( }
Number         : { INT FLOAT }
UnaryExp       : { ID INT FLOAT + - ( }
OptRParams     : { eps ID INT FLOAT + - ( }
UnaryOp        : { + - }
FuncRParams    : { ID INT FLOAT + - ( }
MulExp         : { ID INT FLOAT + - ( }
AddExp         : { ID INT FLOAT + - ( }
RelExp         : { ID INT FLOAT + - ( }
EqExp          : { ID INT FLOAT + - ( }
LAndExp        : { ID INT FLOAT + - ( }
LOrExp         : { ID INT FLOAT + - ( }
ConstExp       : { ID INT FLOAT + - ( }

===== FOLLOW SETS =====
Program       : { $ }
CompUnit      : { $ }
UnitList      : { $ int void const float }
Unit          : { $ int void const float }
Decl          : { $ ID INT FLOAT int void return const float if + - ( { } ;
}
ConstDecl     : { $ ID INT FLOAT int void return const float if + - ( { } ;
}
VarDecl       : { $ ID INT FLOAT int void return const float if + - ( { } ;
}
ConstDefList  : { ; }
BType        : { ID main }
ConstDef     : { ; , }
ConstInit    : { ; , }
VarDefList   : { ; }
VarDef       : { ; , }
InitVal      : { ; , }
FuncDef      : { $ int void const float }
FuncName     : { ( }
OptFParams   : { ) }
FuncType     : { ID main }
FuncFParams  : { ) , }
FuncFParam   : { ) , }

```

```

Block      : { $ ID INT FLOAT int void return const float if else + - ( {
} ; }
BlockItemList : { ID INT FLOAT int return const float if + - ( { } ; }
BlockItem    : { ID INT FLOAT int return const float if + - ( { } ; }
Stmt        : { ID INT FLOAT int return const float if else + - ( { } ; }
OptExp      : { ; }
Exp         : { ) ; , }
Cond        : { ) }
LVa1       : { + - * / % = > < == <= >= != && || ) ; , }
PrimaryExp   : { + - * / % > < == <= >= != && || ) ; , }
Number      : { + - * / % > < == <= >= != && || ) ; , }
UnaryExp    : { + - * / % > < == <= >= != && || ) ; , }
OptRParams  : { ) }
UnaryOp     : { ID INT FLOAT + - ( {
FuncRParams : { ) , }
MulExp     : { + - * / % > < == <= >= != && || ) ; , }
AddExp     : { + - > < == <= >= != && || ) ; , }
RelExp     : { > < == <= >= != && || ) ; , }
EqExp      : { == != && || ) ; , }
LAndExp    : { && || ) ; , }
LOrExp     : { || ) ; , }
ConstExp   : { ; , }

```

- SLR 表构建信息，132个状态，还没实现输出
- 完整的分析过程（PARSING PROCESS 表格）

如果lex传过来的有不合法的源输入，会有详细提示：

```

94 [3] Loading Tokens from ../lex/output/lex2.txt...
95 [LEX DEBUG] Unknown token at source line 116: category='OPzzzzzzzzz', val='14', text='='
96 Loaded 130 tokens.

```

对于error的类型，会有输出信息然后跳过，不影响后面语句继续解析：

```

Syntax > output > 2out.txt
603 [词法错误] 行号 20, 列号 1: 无法识别的字符 '^'
604 |         | 该字符将被跳过，继续分析后续内容。
605 |
606 [词法错误] 行号 20, 列号 3: 无法识别的字符 '!'
607 |         | 该字符将被跳过，继续分析后续内容。
608 |
609 [词法错误] 行号 20, 列号 5: 无法识别的字符 '@'
610 |         | 该字符将被跳过，继续分析后续内容。

```

对于语法错误，会输出详细的错误信息，当前状态，以及期望符号集，可以帮助检查语法错误

```

612 ===== 语法错误详情 =====
613 [错误位置] 行号: 126
614 [错误符号] 文本: '!='
615 [符号类型] ID: 25, 原始类别: 'OP', 符号名: '!='
616 [当前状态] State 128
617 [期望符号] 在当前状态 128 下，可以接受以下符号:
618 |         | 'ID' 'INT' 'FLOAT' 'int' 'return'
619 |         | 'const' 'float' 'if' 'else' '+'
620 |         | '-' '(' '{' '}' ';'
621 [错误原因] 语法分析阶段: 当前符号 '!=' (类型: !=) 在当前语法状态下不被接受
622 =====
623
624 Syntax Error at line 126: Unexpected token '!='
625 |         | Reduction sequence (with error) written to output/syntax2.txt

```

PARSING PROCESS 表格格式:

Step	State	Input	Action
1	0	int	Reduce 4 (UnitList -> ...)
2	2	int	Shift 3
...			

- 语法生成树

```
===== SYNTAX TREE =====
CompUnit
  UnitList
    UnitList
      UnitList
        Unit
          Decl
            VarDecl
              BType
                int (int)
              VarDefList
                VarDef
                  ID (a)
                  = (=)
                  InitVal
                    Exp
                      LOrExp
                        LAndExp
                          EqExp
                            RelExp
                              AddExp
                                MulExp
                                  UnaryExp
                                    PrimaryExp
                                      Number
                                        INT (10)
                                ; (;)
                  Unit
                    FuncDef
                      BType
                        int (int)
                      FuncName
                        main (main)
                      ( (())
                      OptFParams
                    ) (())
                    Block
                      { ({)
                      BlockItemList
                        BlockItemList
                          BlockItem
                            Stmt
                              LVal
                                ID (a)
                                = (=)
```

```

Exp
  LOrExp
    LAndExp
      EqExp
        RelExp
          AddExp
            MulExp
              UnaryExp
                PrimaryExp
                  Number
                    INT (10)

; (;)
BlockItem
  Stmt
    return (return)
  OptExp
    Exp
      LOrExp
        LAndExp
          EqExp
            RelExp
              AddExp
                MulExp
                  UnaryExp
                    PrimaryExp
                      Number
                        INT (0)

; (;)
} (})

```

！ 所以，不管你遇到了什么问题，都可以在日志out文件里先找找答案，中间代码生成如果需要调整语法器，有问题要修改可以借助out日志文件

2. 规约序列 (syntax*.txt)

这个是用来交作业的。就是从out里提取的

位置: Syntax/output/syntax1.txt ~ syntax4.txt

格式:

```
[序号][TAB][栈顶符号]#[面临输入符号][TAB][执行动作]
```

执行动作类型:

- move - 移进 (Shift)
- reduction - 归约 (Reduce)
- accept - 接受
- error - 错误

示例:

```

1  int#int move
2  bType#int  reduction
3  Ident#a move
4  =#= move
5  IntConst#10 move

```

3. 语法树 (tree*.txt)

中间代码最好基于这个来写

树是忽略错误符号的，也就是说对于修正过的词法序列生成的，但是对于语法错误，树就没法生成了。

位置: `Syntax/output/tree1.txt` ~ `tree4.txt`

格式:

```

Program
  CompUnit
    UnitList
      Unit
        FuncDef
          BType
            int (int)
          FuncName
            main (main)
        ...

```

说明:

我们以 `1out.txt` 中解析 `int a = 1, b = 2;` 的这部分为例进行解读:

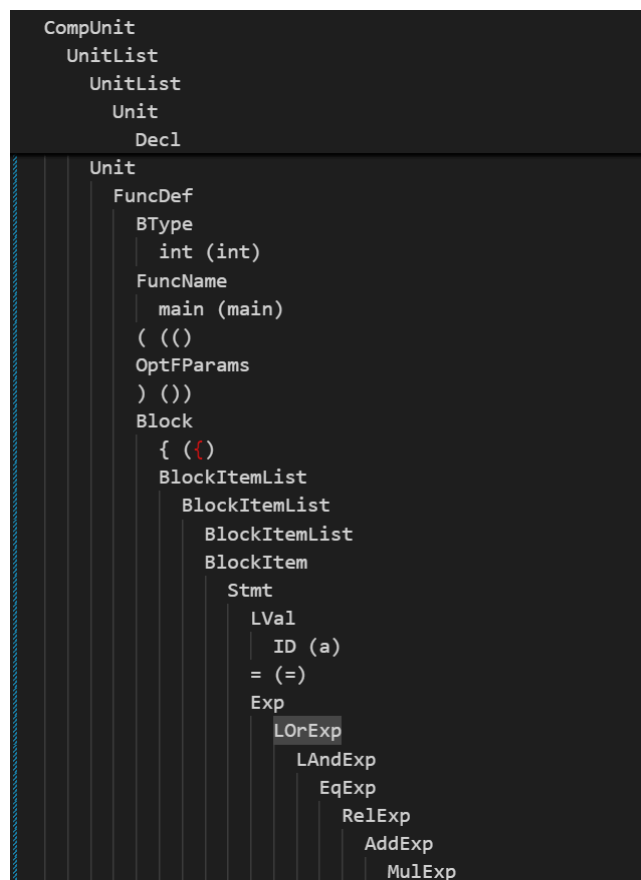
```

VarDecl          <-- 顶层: 这是一个“变量声明”结构
  BType
    int (int)      <-- 类型是 int
  VarDefList      <-- 接下来是一个变量定义列表
    VarDef         <-- 第一个变量定义
      ID (a)        <-- 变量名 a
      = (=)         <-- 赋值号
      InitVal       <-- 初始值
        Exp...      <-- 经过层层表达式推导
          Number
            INT (1)  <-- 值是 1
      , (,)         <-- 逗号分隔
    VarDefList     <-- 列表的递归部分 (第二个变量)
      VarDef
        ID (b)      <-- 变量名 b
        = (=)
        InitVal...
          INT (2)    <-- 值是 2
      ; (;)         <-- 分号结束声明

```

解读方法:

1. **缩进代表层级**: 每一级缩进表示“我是上面那个节点的子节点”。这个在code中查看比较清晰: 如果遍历不方便的话可以处理一下这个树结构的输入。



2. **非终结符 (Non-Terminal)**: 显示符号名称, 如 `VarDecl`, `VarDef`, `Exp`。它们代表一种语法结构 (语句、表达式等)。

3. **终结符 (Terminal)**: 显示符号名 (文本值), 如 `int (int)`, `ID (a)`, `INT (1)`。它们是树的叶子, 对应源代码中实际写出来的字。格式通常是 `Token类型 (Token文本)`。

这棵树的作用: 它告诉你程序的**结构化含义**。看到 `VarDecl`, 编译器就知道“哦, 这里要分配内存了”。看到 `BinaryExp` (在你的树里是 `AddExp` 下有两个子节点), 就知道“这里要做加法运算”。

详细说明

- 词法部分

传递给到语法器的是lex-n.txt文件，类似这样：

```
VoID      <KW,2>
MaIn      <KW,5>
(         <SE,23>
)         <SE,24>
{         <SE,25>
@         <ERROR,1,28>//错误信息是行号和列号
CONST     <KW,4>
InT       <KW,1>
_X1       <IDN,_X1>
=         <OP,14>
42        <INT,42>
,         <SE,28>
y         <IDN,y>
```

- 语法部分

- ☒ **修改文法：**EBNF 转 CFG，作业附录给出的文法使用了 EBNF 符号（`*`，`?`，`|`，`()`），例如规则 2 和规则 17。标准的 SLR 算法只能处理 **上下文无关文法（CFG）**。需要将所有 EBNF 写法展开为递归产生式。**更改完的文法我会整合一下发在 c--产生式.md 文件里。**最后采用的详细的产生式以注释的形式放在了 `Syntax\GrammarAnalyzer.cpp` 文件的 `initGrammar` 内，有几条做了更改，注释也写了更改的原因和方式，第3和7是由于对 `,` 的识别，10和11是由于 `main` 这个KW的定义导致了冲突，17是由于原来的规则不支持括号内的逻辑/关系运算。

相关文件

- 1. c--产生式.md
- 2. GrammarAnalyzer.cpp

- ☒ **对词法传来的lex文件进行解析，建立 Token 映射：**将词法分析器输出的 `<种别, 属性>` 映射为语法分析器能识别的终结符。这部分在 `Syntax\main.cpp` 的第**16-181**行，包含了对 `<ERROR,行号,列号>` 这种特殊情况的处理，对于输入的不符合规则的情况会另外打印解析错误信息，比如这样：`<hello, 88>`，会在 `*out.txt` 日志类文件里打印出来这一行的debug信息的。

相关文件

- 1. main.cpp 映射和解析逻辑在这里
- 2. Parser.cpp 对ERROR的信息输出逻辑在这里

终结符名称	对应词法输出 (种别, 属性值)	说明
ID	<IDN, 字符串>	变量名、函数名
INT_CONST	<INT, 数值>	整数常量
FLOAT_CONST	<FLOAT, 数值>	浮点常量
KW_INT	<KW, 1>	int

终结符名称	对应词法输出 (种别, 属性值)	说明
KW_VOID	<KW, 2>	void
KW_RETURN	<KW, 3>	return
KW_CONST	<KW, 4>	const
KW_MAIN	<KW, 5>	注意: main 是关键字
KW_FLOAT	<KW, 6>	float
KW_IF	<KW, 7>	if
KW_ELSE	<KW, 8>	else
OP_PLUS	<OP, 9>	+
OP_MINUS	<OP, 10>	-
OP_MUL	<OP, 11>	*
OP_DIV	<OP, 12>	/
OP_MOD	<OP, 13>	%
OP_ASSIGN	<OP, 14>	=
OP_GT	<OP, 15>	>
OP_LT	<OP, 16>	<
OP_EQ	<OP, 17>	==
OP_LE	<OP, 18>	<=
OP_GE	<OP, 19>	>=
OP_NEQ	<OP, 20>	!=
OP_AND	<OP, 21>	&&
OP_OR	<OP, 22>	
SE_LPAREN	<SE, 23>	(
SE_RPAREN	<SE, 24>	)
SE_LBRACE	<SE, 25>	{
SE_RBRACE	<SE, 26>	}
SE_SEMICN	<SE, 27>	;
SE_COMMA	<SE, 28>	,

- **✓ productions (项目集规范族的基础) 和 first/followSet 的构建**: 这部分实现在 `Syntax\GrammarAnalyzer.cpp` 中的, `main` 会调用 `GrammarAnalyzer` 类里的 `printSets` 函数在out文件里打印出来所有的 `first/followSet`, 可以有个debug的参考, 如果后面做中间代码生成发现树有问题需要改语法器的话, 可以找找是不是我实现的**两个set和action还是goto表** (表也有打印信息, 但是暂时还没有像follow集这样完整的打印) 有问题。

相关文件:

1. 头文件与数据结构 (`GrammarAnalyzer.h`)

这里我们扩展了之前的枚举, 加入非终结符 (对应我们之前约定的纯 CFG 文法), 并定义了计算所需的集合容器。

2. 算法实现 (`GrammarAnalyzer.cpp`)

这里使用了不动点迭代法。这是实现 FIRST/FOLLOW 最稳健的方法, 特别是在处理左递归文法 (如 `addExp -> addExp + mulExp`) 时, 比纯递归更容易避免死循环。

- **✗ 构造 LR(0) 项目集与 SLR 分析表(Action/Goto Table)**:

这是最复杂的一步, 在 `SLRTable` 类里。它需要完成以下任务:

1. **✓ 构造项目集 (Items)**: 比如 `stmt -> if . ( Cond )`。
2. **✓ 计算闭包 (Closure) 和 转换 (GoTo)**。
3. **✓ 生成分析表**: 这里需要提一下遇到 **移进-归约冲突 (Shift-Reduce Conflict)** 时 (特指 `If-Else`), 这里强制选择 **移进 (Shift)**。
4. **✗ 输出结果**: 将分析表保存为 CSV 或打印出来

相关文件:

`SLRTable.h` `SLRTable.cpp`

构造规范族主要依赖两个函数:

1. `getClosure(I)` (**闭包计算**):

- **原理**: 如果我期待读 `A` (即 `... -> . A ...`), 而 `A` 又能展开成 `B C`, 那么我实际上也在期待读 `B` (即 `A -> . B C`)。
- **代码体现**: `SLRTable::getClosure` 函数。

2. `getGoto(I, X)` (**状态转换**):

- **原理**: 如果我在状态 `I`, 读入符号 `X`, 圆点就会后移, 形成新的状态。
- **代码体现**: `SLRTable::getGoto` 函数。

3. `buildTable()` (**构建主循环**):

- 这个函数通过不断循环 `Closure` 和 `GoTo`, 把 `State 0, State 1...` 一个个找出来, 放入 `states` 数组中。

- **✓ 语法分析驱动程序 (Parser Driver)**: 分析输出规约过程

它的任务是:

1. 读取 Token 流。
2. 维护 **状态栈** 和 **符号栈** (用于构建语法树)。
3. 查表执行 Shift (移进) 或 Reduce (归约)。
4. **构建语法树 (AST)** 并输出归约序列。

负责输出PARSING PROCESS表，为了改bug我们在日志输出的时候选择建立了一个特别详细好读的表，出现问题可以查这个 PARSING PROCESS表，包含Step, State, Input, Action四个信息，以及如果是规约，应该被规约为什么（reduction后面括号里的内容）

相关文件：

1. `Parser.h` 和 `Parser.cpp` 分析器驱动
2. `SyntaxTree.h` 用于生成语法分析树的

- ☒ **规约序列提取**：从上一步的里面提取老师要求的格式的规约序列

相关文件：

1. `ReductionSequenceLogger.cpp`
2. `ReductionSequenceLogger.h`

配置与自定义

修改语法分析器处理的文件

语法分析器每次只能处理一个文件

编辑 `Syntax/main.cpp`，找到以下行：

```
string lexFilename = "../lex/output/lex1.txt";
```

修改为你需要的文件名，例如：

```
string lexFilename = "../lex/output/lex2.txt";
```

添加新的测试用例

1. 添加源文件：

- 在 `lex/test/` 目录下创建新的 `.sy` 文件（如 `test5.sy`）

2. 修改词法分析器：

- 编辑 `lex/Lex_Analysis.cpp` 中的 `lexical_analysis()` 函数
- 添加新文件的处理：`lexcial("test/test5.sy", minDFA, "5");`
- 进行词法分析：

```
##命令
1.cd lex

2.g++ Lex_Analysis.cpp -o Lex_Analysis

3../Lex_Analysis
```

或者干脆用bat程序批处理就行。

3. 修改语法分析器：

- 编辑 `Syntax/main.cpp` 中的 `lexFilename` 变量
- 进行语法分析：

```
1.cd Syntax

2.g++ main.cpp GrammarAnalyzer.cpp SLRTable.cpp Parser.cpp
ReductionSequenceLogger.cpp -o compiler_test

3../compiler_test
```

会生成对应的lex*的syntax, out, tree文件。一次只能跑一个测试用例。

清理构建产物

Windows

```
cd lex
del Lex_Analysis.exe
cd ../Syntax
del compiler_test.exe
cd ..
```

Linux/Mac

```
make clean
```

联系与支持

如有问题，请检查：

1. 编译错误信息
2. 输出日志文件（*out.txt）
3. 源代码中的注释和文档

最后更新：2025年11月28日